

Values and Variables

In the last lesson, you ran small pieces of JavaScript in the browser console. Now we can start looking at what JavaScript code is actually made of.

Most programs move information around. A shopping cart deals with prices and quantities. A login form deals with email addresses and passwords. A video game deals with scores, player names, and whether the game is over.

In JavaScript, we call these pieces of information **values**. A **variable** is a name you give to a value so your code can find it and use it later.

That is the big idea for this lesson. Values are the information. Variables are the names.

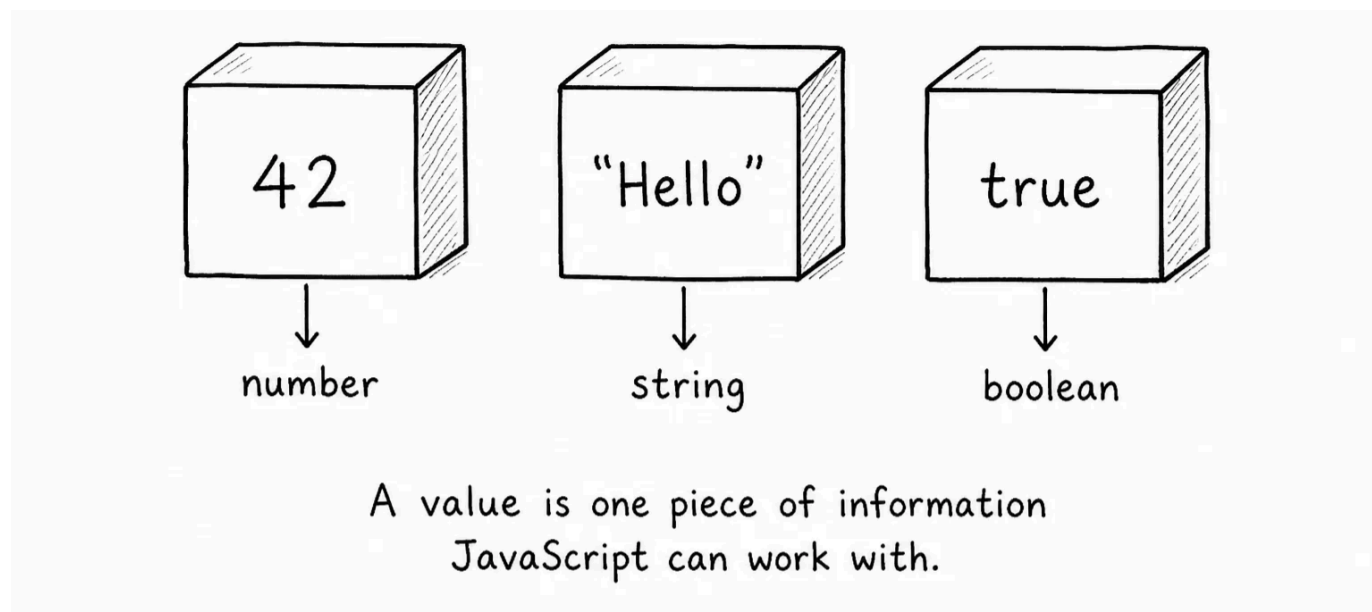
Values are pieces of information

A value is one single piece of data your program can work with. You actually saw a few values in the console during the last lesson:

```
js   
  
42  
"Hello, JavaScript"  
true
```

42 is a number. **"Hello, JavaScript"** is a string of text. **true** is a boolean, which is a value that can only be **true** or **false**.

You can type these values directly into your code. When you write a value directly like this, programmers often call it a **literal**. You do not need to memorize that word right now, but you will see it a lot in documentation.



Common value types

JavaScript has a few different kinds of values. For now, we will focus on the ones you will use every single day.

Value	Type	Used for
"Ava"	string	Text
42	number	Numbers and calculations
true	boolean	Yes/no or on/off decisions
false	boolean	Yes/no or on/off decisions
null	null	Intentionally empty
undefined	undefined	Missing or not set yet

Strings are text. You usually wrap them in quotes:

```
js
```



```
"Welcome back"
```

Numbers are exactly what they sound like. You use them for math:

```
js
```



```
20 + 5
```

Booleans are perfect for things that only have two states:

```
js
```



```
true
```

You will use booleans constantly once your code starts making decisions. Think about it. A user is either logged in or they are not. A form is either valid or invalid. A menu is open or closed.

null and undefined

Both **null** and **undefined** mean "there is no useful value here." But they are used in slightly different ways.

You use **null** when you want your code to specifically say, "I am intentionally leaving this empty."

```
js
```



```
const selectedUser = null;
```

This tells anyone reading the code that no user has been selected yet.

undefined usually means a value is missing, or the computer has not assigned it yet:

```
js
let selectedUser;

console.log(selectedUser);
```

If you run this, the console will show **undefined**. The name exists, but you never actually gave it a value.

Do not stress if this difference feels blurry right now. Remember this practical habit: use **null** when you want to explicitly say "nothing is here." Expect to see **undefined** when something is missing or forgotten.

⚠ **null is not the same as "null"** - **null** is a real JavaScript value. **"null"** is a text string containing the letters n, u, l, and l. The quotes change everything.

Variables give values names

Writing values directly is fine for quick tests. But in real apps, you need to reuse information. That is what variables are for.

A variable simply gives a value a name:

```
js
const message = "Hello, JavaScript";
```

Read that line like this: create a name called **message** , and point it directly at the value **"Hello, JavaScript"** .

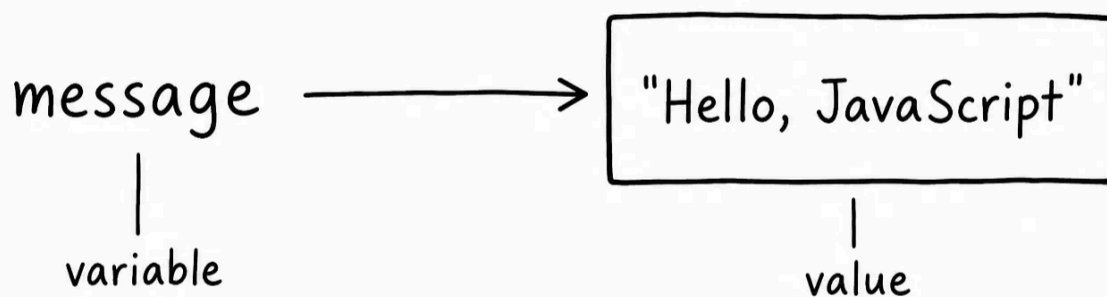
Now you can use that name later in your code:

```
js
const message = "Hello, JavaScript";

console.log(message);
```

The console will print **"Hello, JavaScript"** . JavaScript looks at the name **message** , finds the text connected to it, and prints it out.

```
txt
message -> "Hello, JavaScript"
```



A variable is a name JavaScript uses to find a value.

Why names matter

Variables make your code much easier to read because they explain what the data actually means.

This code works perfectly fine:

js



```
20 + 5
```

But this code actually tells a story:

js



```
const price = 20;  
const tax = 5;  
  
const total = price + tax;  
  
console.log(total);
```

Suddenly, those numbers have meaning. **20** is not a random number. It is the price. **5** is the tax. **total** is the result of adding them up.

Naming things well is a huge part of being a good developer. A name like **total** is clearer than **x**. A name like **isLoggedIn** is clearer than **status**.

💡 **Use names that explain the job** - If your variable name explains why the data exists, your code becomes easier to read. This is useful when you come back to your own code three months later.

Use `const` by default

You will create most of your variables using `const` :

js



```
const userName = "Ava";
```

const is short for constant. It means the variable name cannot be reassigned to a different value later on.

js



```
const userName = "Ava";  
  
userName = "Mia";
```

If you try to run that, JavaScript will throw an error. It stops you because you told it **userName** was a constant.

This is a good thing. It protects you from accidentally pointing a name somewhere else halfway through your file. If a name should keep pointing to the same value, use **const** .

Use **let** when the value needs to change

Sometimes a variable does need to change. Maybe it is a score in a game, or a counter. For that, use **let** .

js



```
let score = 0;  
  
score = score + 10;  
  
console.log(score);
```

The first line creates **score** and sets it to **0** . The second line looks at the old score, adds **10** , and saves the new result back into **score** . The console prints **10** .

Here is the golden rule for variables:

txt



Use `const` when the name should keep pointing to the same value.
Use `let` when you know the name will need to point to a new value later.

In the real world, you will use **`const`** way more often than **`let`**. Start with **`const`**. If you realize later that you need to change the value, you can easily swap it to **`let`**.

Naming rules

Variable names have a few strict rules.

They can contain letters, numbers, underscores (`_`), and dollar signs (`$`). But they cannot start with a number.

These are perfectly fine:

js



```
const user1 = "Ava";  
const user_name = "Ava";
```

This will cause a syntax error:

js



```
const 1user = "Ava";
```

Also, most JavaScript developers use a style called **camelCase** for naming. That means the first word is all lowercase, and any words attached after it start with a capital letter:

js




```
const userName = "Ava";  
const isLoggedIn = true;  
const totalPrice = 25;
```

Finally, names are case-sensitive. `userName` and `username` are completely different variables to JavaScript.

What to practice

Your goal right now is not to memorize every single data type. Your goal is to build a solid mental model:

txt



value: one single piece of information
variable: a name attached to a value
const: creates a name that cannot be reassigned
let: creates a name that can be reassigned

Once that clicks, everything else gets easier. Decisions rely on booleans. Functions take in values and return new values. Objects and arrays group values together.



AI Tutor

Explain JavaScript values and variables using a simple shopping cart example. Include strings, numbers, booleans, null, const, and let.



Try it

Open your browser console and run this chunk of code:

js



```
const productName = "Notebook";  
const price = 12;  
const quantity = 3;
```

```
const total = price * quantity;

console.log(productName);
console.log(total);
```

You should see **Notebook** and **36** print out.

Now try a value that needs to change:

```
js 📄

let cartItems = 3;

cartItems = cartItems + 1;

console.log(cartItems);
```

You should see **4** .

If your console complains that a name has already been declared, refresh the webpage. The browser remembers variables from earlier. Refreshing gives you a clean slate.

Finally, try this:

```
js 📄

const selectedUser = null;

console.log(selectedUser);
```

The console should print **null** . You explicitly told it to hold nothing.

< 0 / 7 >

👉 Knew 0 Items 🌟 Learnt 0 Items ▶ Skipped 0 Items 🔄 ResetProgress

Test Your Understanding

What is a value?

[Click to Reveal the Answer](#)

✓ Already Know that

✳ Didn't Know that

⏩ Skip Question

TRACK YOUR PROGRESS

Finished this lesson?

Mark it as done to track your progress through the pack.

Mark as Done